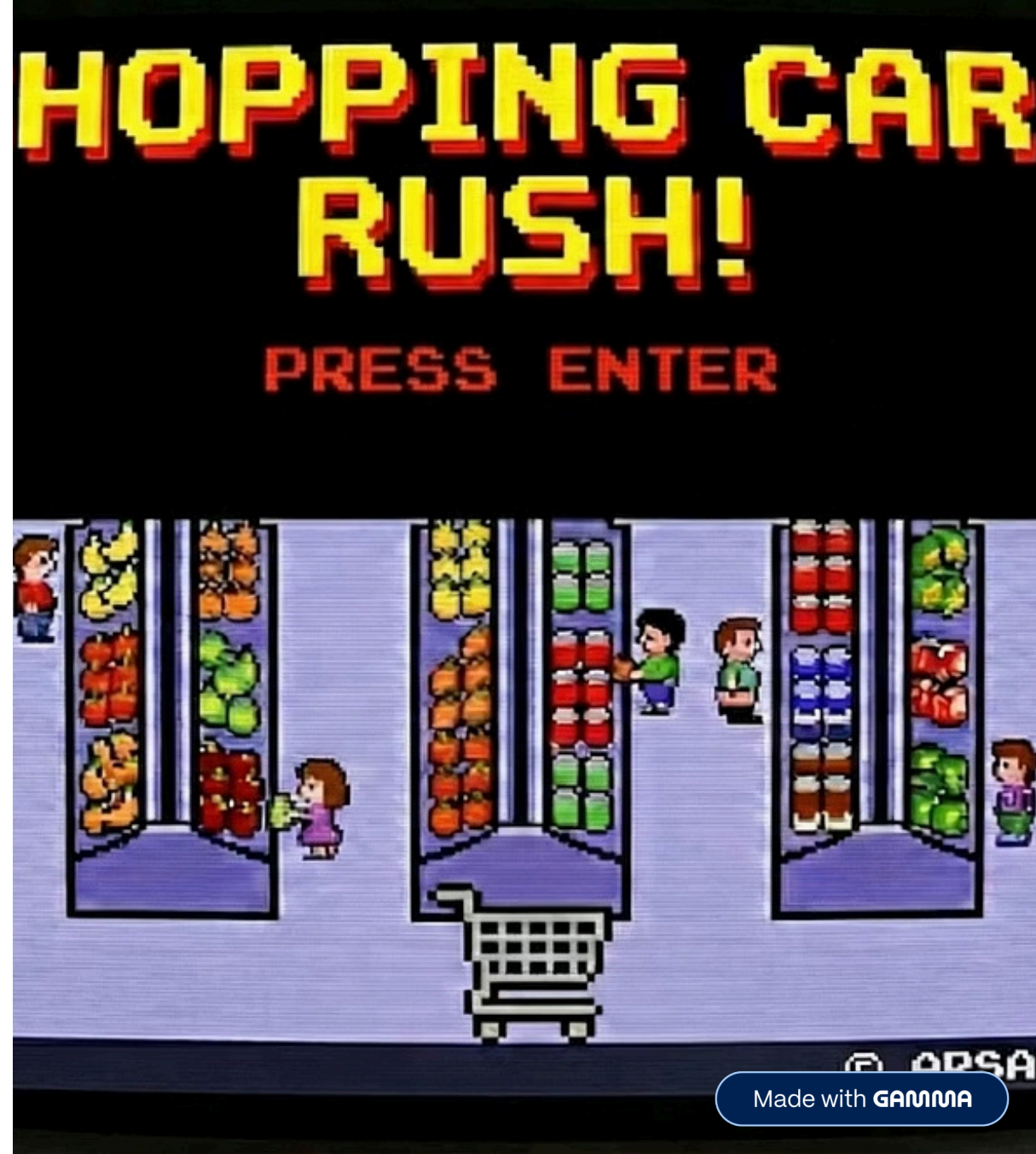


Shopping Cart Rush

A VGA-Based Arcade Game for FPGA

Players navigate a shopping cart through a busy grocery store, dodging shoppers whilst collecting coins to maximise their score.

by Aryan and Samarth



Project Overview and Inspiration

Game Concept

Shopping Cart Rush combines classic arcade gameplay with the familiar setting of a supermarket. Players control a shopping cart rushing down the aisle, avoiding collisions with other shoppers whilst collecting coins to increase their score.

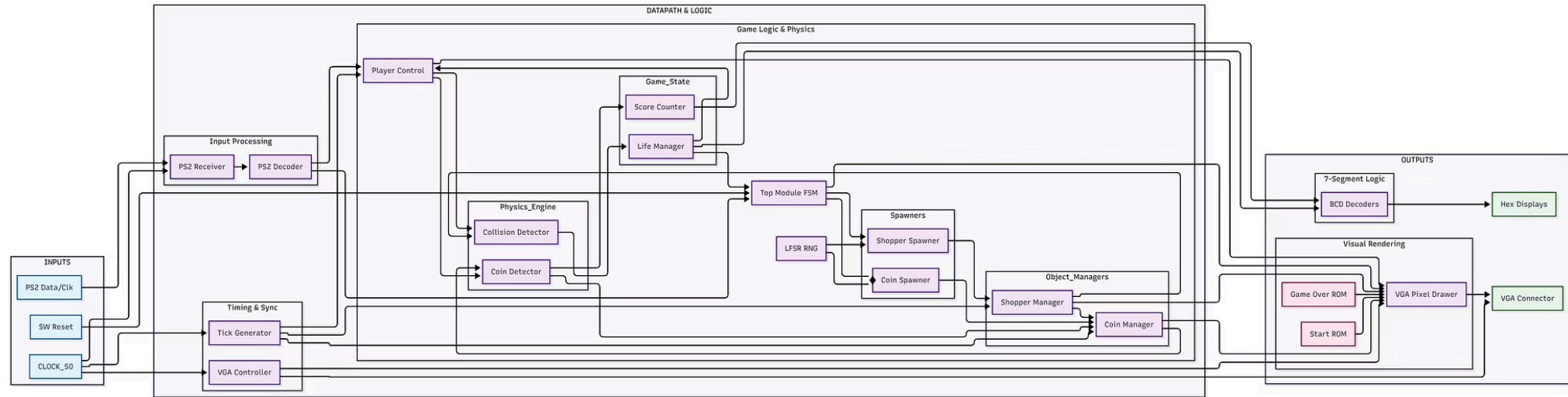
The game features dynamic difficulty through random lane spawning and variable shopping cart speeds, creating an challenging experience as the game progresses which tests the players reflexes.

Design Philosophy

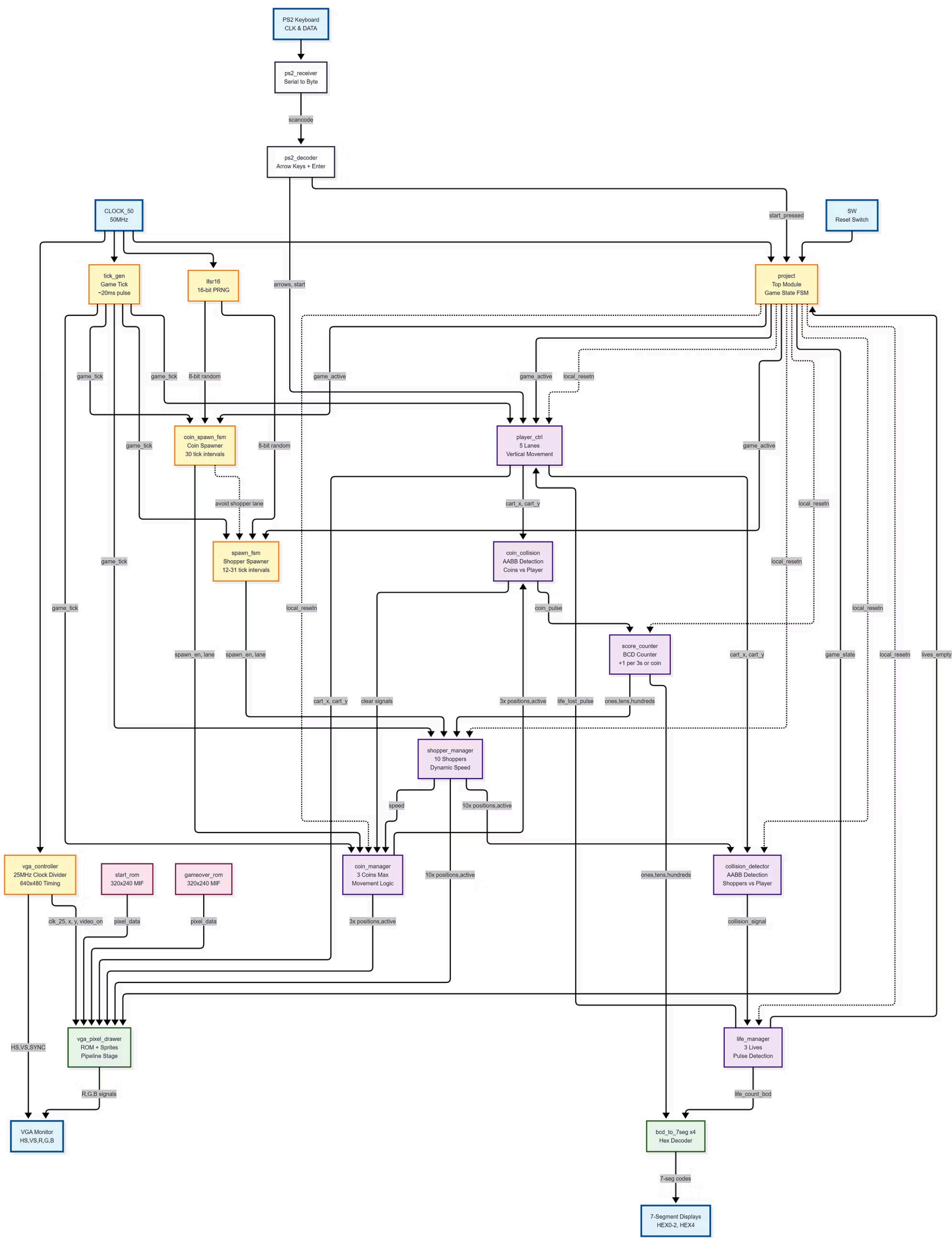
Inspired by classic endless runner games and retro arcade titles, we aimed to create a game that would be both technically impressive on FPGA hardware and entertaining to play.

The supermarket theme adds a relatable, humorous twist to the traditional obstacle-avoidance genre.

High-Level System Architecture



Low-Level System Architecture



Core Component Architecture



VGA Display Controller / Pixel Drawer

Manages synchronisation signals, pixel clock generation, and coordinate mapping for 640×480 resolution. Renders all visual elements including sprites and background.



Player Input Control

Handles player position tracking across five lanes, and responds to PS2 keyboard inputs for smooth left/right/up/down movement control.



Spawn FSM / Shopper Manager / Coin Manager

Uses 16-bit Linear Feedback Shift Register (LFSR) for pseudo-random lane assignment. Controls vertical movement of shopper objects and coin objects and manages multiple of them simultaneously.



Collision Detection

Implements pixel-perfect overlap detection between player cart and shopper sprites. Triggers life manager and game over sequence upon collision events.



Scoring System

Tracks survival time and coin collection. Updates score display in real-time and stores scores in 7-segment displays.



Game State Manager

Controls transitions between start screen, active gameplay, and game-over states. Manages reset logic.

Development Timeline and Milestones

Week 9: Foundation Phase

Project Proposal Submission

- Implemented VGA display system with 640×480 output
- Created player graphic and static background graphics
- Enabled FPGA key-based movement control
- Established basic game framework and structure

Week 11: Enhancement Phase (Milestone 2)

Scoring and Difficulty Systems

- Integrated scoring logic combining time and item collection
- Implemented variable speed system for shopper movement depending on the player's score
- Added game reset functionality post-collision

1

2

3

4

Week 10: Core Gameplay (Milestone 1)

Dynamic Elements Integration

- Implemented LFSR-based random lane spawning system
- Added vertical movement for shopper objects
- Developed collision detection through overlap algorithm
- Refined lane logic and ensured smooth gameplay mechanics

Week 12: Final Integration

Presentation Preparation

- Complete system integration and final testing
- Added game start and game-over screen and score display enhancements
- Prepared demonstration and presentation slides
- Documented all components and conducted final polish

Game in Action



YouTube



shopping cart rush demo

ECE241 project 2025

The video showcases the game running on the FPGA board.

Challenges and Solutions

VGA Timing Synchronisation

Challenge: Achieving stable VGA output without player sprite glitching while moving left and right proved difficult initially.

Solution: Implemented discrete fixed lane movements within five lanes instead of continuous movement for the player.

VGA Signal Glitching & Timing Violations

Challenge: Random colored lines and flickering occurred because complex combinational pixel-color logic couldn't settle within a 25 MHz cycle, causing timing violations and runt pulses.

Solution: We added an output pipeline stage, registering the next pixel's color before driving the VGA pins, which stabilized timing and eliminated glitches.

Input Responsiveness (One Movement)

Challenge: Key-press pulses from the 50 MHz PS/2 controller lasted only one cycle, so most presses were missed between the game's ~60 Hz ticks, causing unresponsive controls.

Solution: We added input latching: each key press sets a request flag that persists until the next game_tick, ensuring every press is captured and processed exactly once.

Integer Overflow in Speed Logic

Challenge: Movement stopped at a score of 36 because the 4-bit speed register overflowed at 16, wrapping to 0 and freezing the shoppers.

Solution: We expanded the speed register to 10 bits and added saturation logic to cap the value, preventing overflow and maintaining correct movement physics.

Technical Achievements

640×480

VGA Resolution

Full standard VGA output with 60Hz refresh rate

5

Movement Lanes

Dynamic lane system for strategic gameplay

+1

Score and Speed

Speed increases as score increases

2

Object Types

Collectible coins and Dodge shoppers

Future Enhancements

Planned Improvements

01

Audio Integration

Add sound effects for collisions, item collection, and background music to enhance immersion

02

Power-Up System

Implement temporary invincibility after collision, speed boosts, and score multipliers for strategic depth

03

Difficulty Options

Have three difficulty options at the start screen: easy, medium and hard

04

High Score Persistence

Store top scores in FPGA memory for competitive gameplay across sessions

Advanced Features

01

Multiple Game Modes

Add timed challenges, endless mode, and multiplayer competitive options

02

Enhanced Graphics

Create animated sprites, parallax scrolling backgrounds, and particle effects

03

Obstacle Variety

Introduce new obstacle types such as shopping trolleys, spills, and promotional displays

04

Level Progression

Design multiple store sections with unique visual themes and gameplay mechanics

Work Distribution

Aryan: Hardware & Graphics

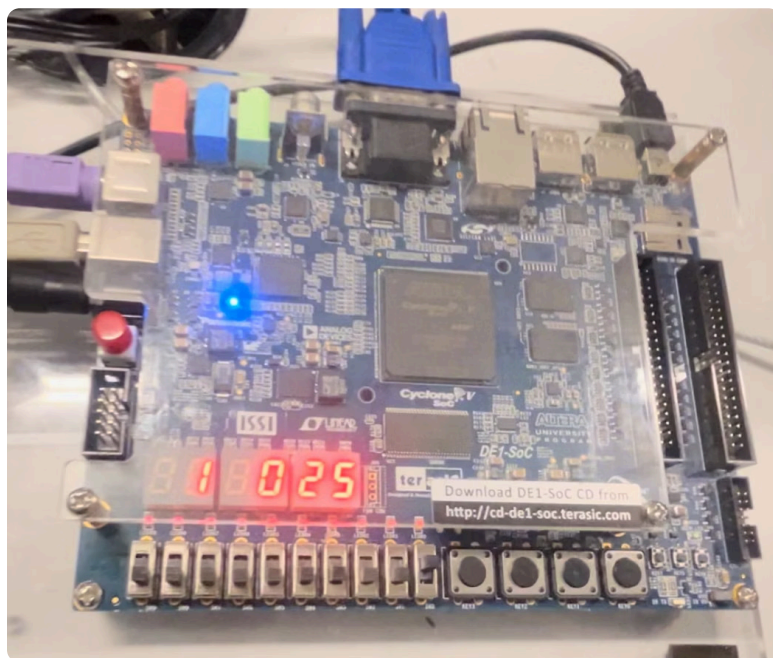
- project (top module)
- VGA (vga_controller, vga_pixel_drawer, rom)
- PS2 keyboard (ps2_receiver, decoder)
- Collision (collision_detector, life_manager)
- Score (score_counter, bcd_to7seg)

Samarth: Game Logic & Systems

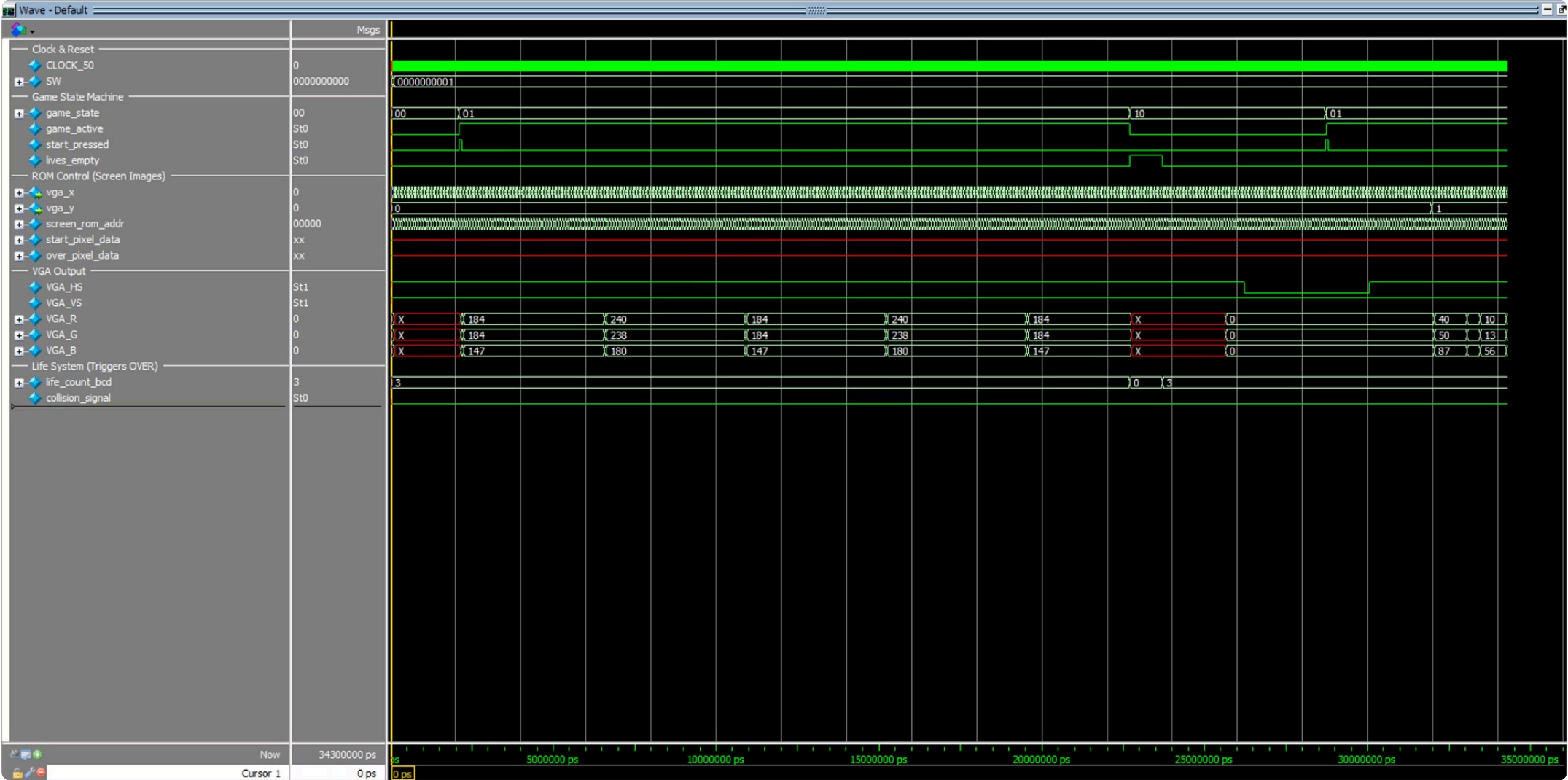
- Project (top module)
- Player (player_ctrl)
- Randomization algorithm (lfsr16, spawn_fsm, coin_spawn)
- Object (shopper_manager, coin_manager, coin_collision)
- Game Tick (tick_gen)

Collaborative Efforts: Both team members contributed equally to project planning, milestone reviews, debugging sessions, and presentation preparation. Regular pair programming sessions ensured knowledge sharing and code quality throughout development.

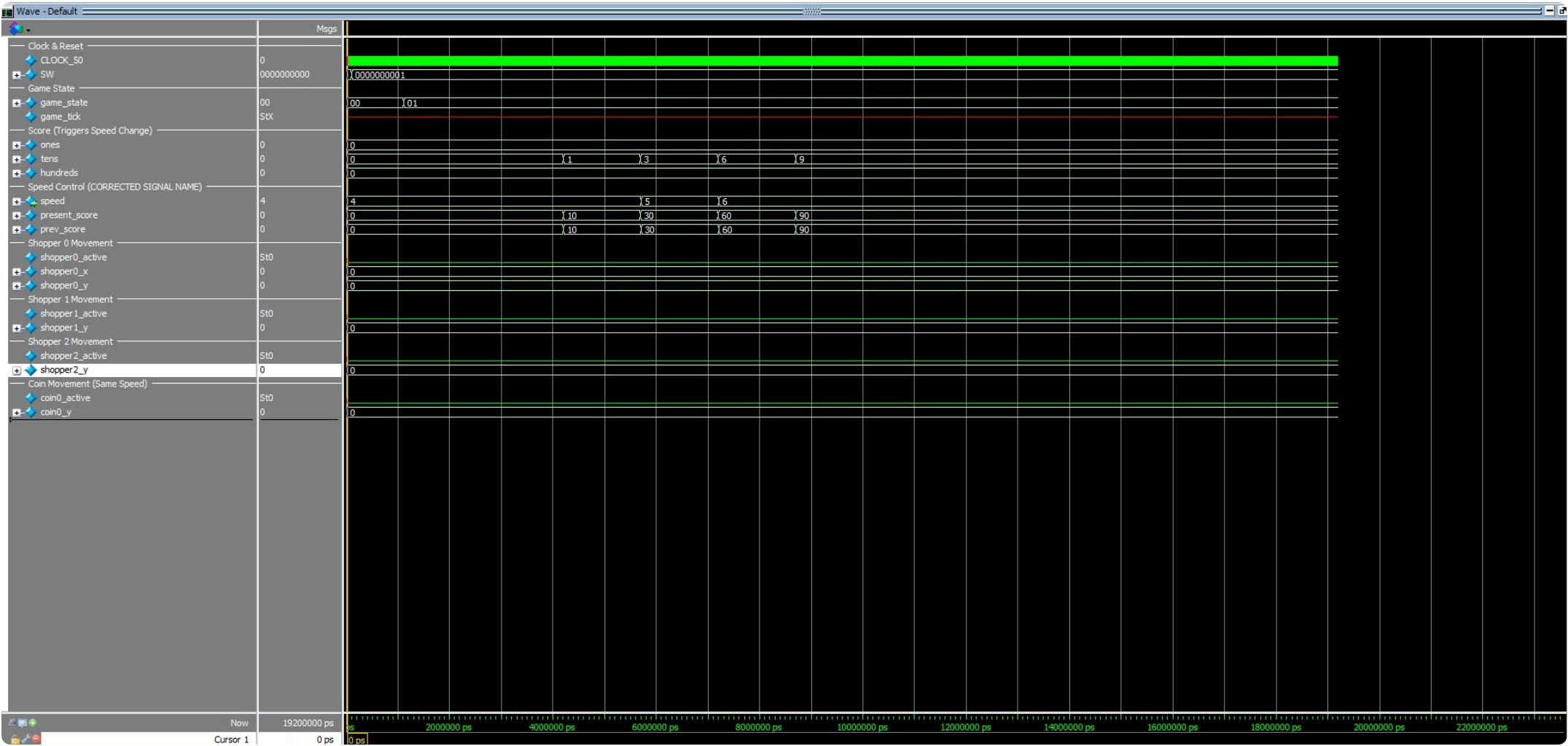
Appendix A: Game's Images



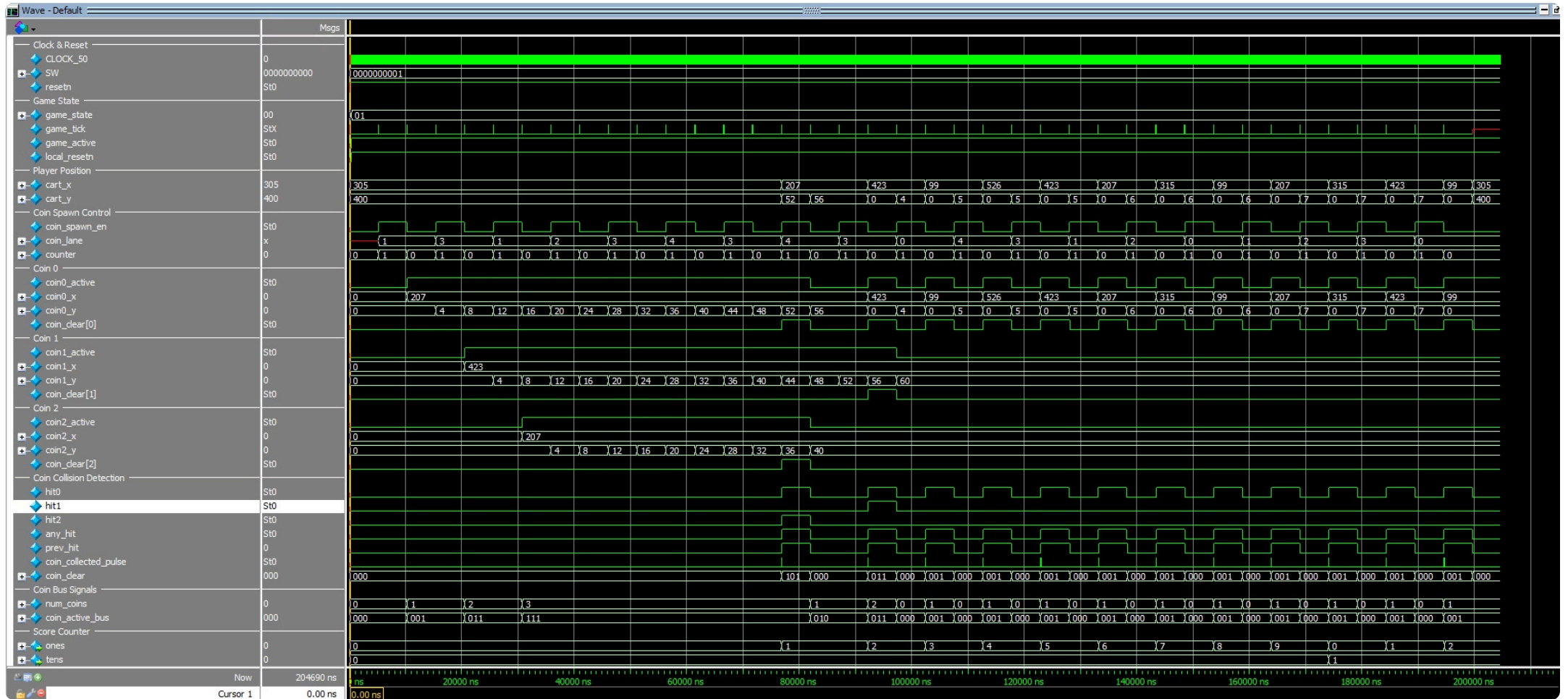
Appendix B: Model Sim waveform for game state



Appendix C: Model Sim waveform for increasing speed of game



Appendix D: Model Sim waveform for coin objects



Appendix E: Model Sim waveform for coin objects

